

Integración Continua con GitHub Actions

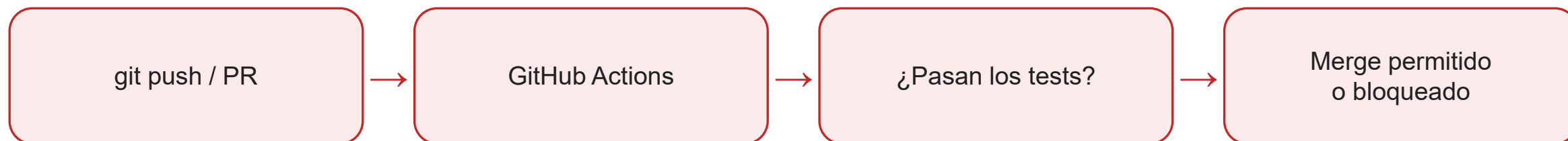
Apartado 5 — CI, workflows, jobs, steps y Actions

9.1 — ¿Qué problema resuelve la integración continua?

Sin CI: cuatro personas trabajan en el mismo repositorio. Cada una hace merge cuando termina. Nadie sabe si el código del compañero rompe los tests que otro escribió la semana pasada. El error se descubre días después — o en producción.

Con integración continua (CI)

La CI ejecuta comprobaciones automáticas (compilar, tests, estilo) cada vez que alguien propone un cambio. Si algo falla, el sistema avisa antes de que el código llegue a main.



9.1 — Ventaja clave y ejemplo de este apartado

El equipo puede trabajar más rápido porque confía en que si el pipeline es verde, el código funciona. Sin CI, cada merge es un riesgo. Con CI, es una rutina segura.

Ejemplo que usaremos en este apartado

- Clase Calculadora con 4 operaciones: sumar, restar, multiplicar, dividir
- 5 tests unitarios en CalculadoraTest que verifican cada operación
- Workflow de CI que compila y ejecuta los tests automáticamente en cada push a main
- Introduciremos un bug deliberado para ver cómo GitHub Actions lo detecta

9.2 — GitHub Actions: eventos que disparan un workflow

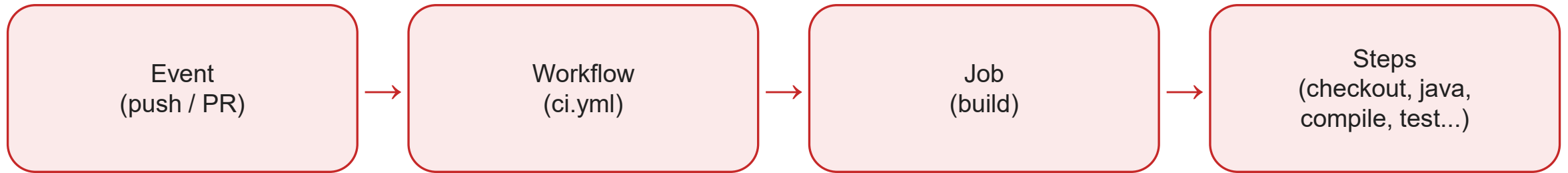
GitHub Actions permite definir workflows que se ejecutan en respuesta a eventos del repositorio, en la nube de GitHub, sin instalar nada en tu máquina.

Evento	Cuándo ocurre
push	Alguien sube commits a una rama
pull_request	Se abre, actualiza o cierra una PR
schedule	Un temporizador (como un cron job)
workflow_dispatch	El usuario lo activa manualmente desde la web
release	Se publica una nueva versión del proyecto

9.2 — Conceptos clave de GitHub Actions

Concepto	Qué es
Workflow	El fichero YAML que describe qué hacer y cuándo. Vive en <code>.github/workflows/</code> .
Event	El disparador que pone en marcha el workflow (push, pull_request...).
Job	Un bloque de pasos que se ejecuta en una máquina virtual.
Step	Una acción concreta dentro de un job: comando de shell o Action del Marketplace.
Runner	La máquina virtual donde corre el job. GitHub ofrece Ubuntu, Windows y macOS.
Action	Tarea reutilizable publicada en el Marketplace. Se invoca con <code>uses:</code> .

9.2 — Jerarquía: Event → Workflow → Job → Steps



En el proyecto Calculadora: el event es push a main → activa el workflow ci.yml → ejecuta el job build → que tiene 5 steps en orden.

- Un workflow puede tener varios jobs (por defecto se ejecutan en paralelo).
- Los steps de un job se ejecutan en orden. Si uno falla, el job se detiene.
- El runner es una máquina virtual Ubuntu que arranca vacía en cada ejecución.

9.3 — Dónde va el workflow en el proyecto

```
mi-proyecto-ci/  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
├── src/  
│   ├── main/java/com/ejemplo/  
│   │   └── Calculadora.java  
│   └── test/java/com/ejemplo/  
│       └── CalculadoraTest.java  
└── pom.xml
```

- name — nombre visible en el panel Actions
- on — eventos que activan el workflow
- jobs — uno o más bloques de trabajo
- runs-on — tipo de runner (ubuntu-latest)
- steps — pasos en orden, uno tras otro
- uses — Action del Marketplace
- run — comando de shell directo

El fichero `.github/workflows/ci.yml` es suficiente para activar CI en cualquier repositorio de GitHub.

9.3 — El workflow ci.yml: cabecera (name, on, jobs)

```
# .github/workflows/ci.yml
name: CI — Build y Tests      # nombre en el panel Actions

on:                            # qué eventos lo activan
  push:                         # solo en push a main
    branches: [main]
  pull_request:                # en cualquier PR hacia main
    branches: [main]

jobs:                           # bloques de trabajo
  build:                       # nombre del job (lo eliges tú)
    runs-on: ubuntu-latest     # máquina virtual Ubuntu
```

- El workflow se activa en dos situaciones: push directo a main y apertura de una PR.
- runs-on: ubuntu-latest — Linux gratuito, arranca en segundos, parte de cero cada vez.

9.3 — El workflow ci.yml: steps

```
steps:
  - name: Descargar el código fuente
    uses: actions/checkout@v4

  - name: Instalar Java 21 (Temurin)
    uses: actions/setup-java@v4
    with:
      java-version: '21'
      distribution: 'temurin'
      cache: 'maven'

  - name: Compilar el proyecto
    run: mvn compile -B

  - name: Ejecutar los tests
    run: mvn test -B

  - name: Guardar el informe de tests
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: informe-tests
      path: target/surefire-reports/
```

9.4 — El proyecto Calculadora: Calculadora.java

Clase con 4 operaciones. La división lanza una excepción si el divisor es cero, lo que permite testear también el comportamiento de error.

```
package com.ejemplo;

public class Calculadora {

    public int sumar(int a, int b) { return a + b; }
    public int restar(int a, int b) { return a - b; }
    public int multiplicar(int a, int b) { return a * b; }

    public double dividir(double a, double b) {
        if (b == 0) {
            throw new IllegalArgumentException("No se puede dividir por cero");
        }
        return a / b;
    }
}
```

9.4 — El proyecto Calculadora: CalculadoraTest.java

5 tests con JUnit 5. Cada método `@Test` comprueba una operación. Maven los ejecuta automáticamente con `mvn test`.

```
class CalculadoraTest {
    private final Calculadora calc = new Calculadora();

    @Test void sumarDosPositivos() {
        assertEquals(5, calc.sumar(2, 3));
    }
    @Test void restarResultadoNegativo() {
        assertEquals(-1, calc.restar(2, 3));
    }
    @Test void multiplicarPorCero() {
        assertEquals(0, calc.multiplicar(7, 0));
    }
    @Test void dividirNormal() {
        assertEquals(2.5, calc.dividir(5, 2));
    }
    @Test void dividirPorCeroLanzaExcepcion() {
        assertThrows(IllegalArgumentException.class, () -> calc.dividir(10, 0));
    }
}
```

9.4 — Qué hace cada step con la Calculadora

Step 1 — checkout@v4

- El runner arranca vacío. Clona el repo: descarga Calculadora.java, los tests y el pom.xml.

Step 2 — setup-java@v4

- Instala JDK 21 (Temurin). Cachéa ~/.m2 para no descargar JUnit en cada run.

Step 3 — mvn compile -B

- Compila Calculadora.java y CalculadoraTest.java. Si hay error de sintaxis, el workflow para aquí.

Step 4 — mvn test -B

- Lanza JUnit con los 5 @Test. Si sumar devuelve el valor equivocado, falla aquí.

Step 5 — upload-artifact (if: always())

- Guarda el informe de Surefire aunque los tests hayan fallado. Sin if:always() el informe se perdería justo cuando más lo necesitas.

9.4 — Primer run en GitHub Actions

Tras el primer push a main, el workflow CI — Build y Tests aparece en la pestaña Actions en segundos.

Open menu

Code Issues Pull requests Agents **Actions** Projects Wiki More ▾

All workflows ▾

New workflow

Showing runs from all workflows

🔍 Filter workflow runs

1 workflow run

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

✓ Proyecto Calculadora con CI

CI — Build y Tests #1: Commit [578e3e4](#) pushed by [aitorventura](#)

📅 1 minute ago

🕒 26s main

...

9.5 — El panel Actions: vista general

-  Naranja (girando) — el workflow está ejecutándose en ese momento
-  Verde — todos los steps han pasado correctamente
-  Rojo — algún step ha fallado (tests, compilación o YAML incorrecto)

All workflows ▾

[New workflow](#)

Showing runs from all workflows

3 workflow runs

Workflow ▾Event ▾Status ▾Branch ▾Actor ▾

Corrige bug en sumar

CI — Build y Tests #3: Commit [46f2bb7](#) pushed by [aitorventura](#)

now

In progress

main

Bug deliberado para ver fallo en CI

CI — Build y Tests #2: Commit [870b80b](#) pushed by [aitorventura](#)

2 minutes ago

18s

main

Proyecto Calculadora con CI

CI — Build y Tests #1: Commit [578e3e4](#) pushed by [aitorventura](#)

5 minutes ago

26s

main

9.5 — Vista de un run: el job build

Al hacer clic en un run concreto se ven todos los jobs. En el proyecto Calculadora hay un solo job llamado build con sus 5 steps y la duración de cada uno.

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) **Actions** [Projects](#) [Wiki](#) [More](#) ▾

[← CI — Build y Tests](#)

 **Proyecto Calculadora con CI #1** 

Summary ▾

Triggered via push 1 minute ago

 **aitorventura** pushed  [578e3e4](#) [main](#)

Status	Total duration	Artifacts
Success	<u>26s</u>	<u>1</u>

ci.yml
on: push

 **build** 23s

9.5 — Vista de un step: log de Maven

Al expandir el step "Ejecutar los tests" se ve la salida de Maven: Tests run: 5, Failures: 0. Los 5 tests de CalculadoraTest han pasado.

build

succeeded 1 minute ago in 23s

✓ Ejecutar los tests6s

251 [INFO] T E S T S
252 [INFO] -----
253 [INFO] Running com.ejemplo.CalculadoraTest
254 [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.049 s -- in
com.ejemplo.CalculadoraTest
255 [INFO]
256 [INFO] Results:
257 [INFO]
258 [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
259 [INFO]
260 [INFO] -----
261 [INFO] BUILD SUCCESS
262 [INFO] -----
263 [INFO] Total time: 4.863 s
264 [INFO] Finished at: 2026-06-30T18:33:39Z
265 [INFO] -----

> ✓ Guardar el informe de tests1s

> ✓ Post Instalar Java 21 (Temurin)0s

> ✓ Post Descargar el código fuente1s

> ✓ Complete job0s

9.6 — Cuando el workflow falla: error en el código

Bug deliberado: cambiamos `return a + b` por `return a - b` en `sumar()`. El test `sumarDosPositivos` espera 5 pero obtiene -1. GitHub bloquea el merge.

```
build
failed now in 14s

v x Ejecutar los tests 3s
50 [INFO]
51 Error: Failures:
52 Error:   CalculadoraTest.sumarDosPositivos:12 expected: <5> but was: <-1>
53 [INFO]
54 Error: Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
55 [INFO]
56 [INFO] -----
57 [INFO] BUILD FAILURE
58 [INFO] -----
59 [INFO] Total time: 1.841 s
60 [INFO] Finished at: 2026-06-30T18:36:39Z
61 [INFO] -----
62 Error: Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test
    (default-test) on project mi-proyecto-ci-calculadora: There are test failures.
63 Error:
64 Error: Please refer to /home/runner/work/mi-proyecto-ci-calculadora/mi-proyecto-ci-
    calculadora/target/surefire-reports for the individual test results.
65 Error: Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and
    [date].dumpstream.
66 Error: -> [Help 1]
67 Error:
68 Error: To see the full stack trace of the errors, re-run Maven with the -e switch.
69 Error: Re-run Maven using the -X switch to enable full debug logging.
70 Error:
71 Error: For more information about the errors and possible solutions, please read the
    following articles:
```

9.6 — Tres tipos de fallo en un workflow

Tipo de error	Ejemplo real	Solución
Error en el código (lo más frecuente)	expected: <5> but was: <-1> en sumarDosPositivos	Corrige el código, commit y push
Error en el workflow	Unable to resolve action actions/setup-java@v99	Corrige el fichero .yml y vuelve a hacer push
Error de entorno	JAVA_HOME is not set java not in PATH	Añade el step setup-java@v4 antes del que falla

Si los tests pasan en tu máquina pero fallan en el runner: versión de Java distinta, dependencia que no se instala, o un fichero local que no está en el repositorio. El runner parte de cero cada vez.

9.7 — El Marketplace de Actions

Miles de Actions reutilizables publicadas por GitHub y la comunidad. En el proyecto Calculadora ya usamos tres de las más habituales.

Action	Para qué sirve
actions/checkout@v4	Clona el repositorio en el runner. Siempre el primer step.
actions/setup-java@v4	Instala un JDK concreto (Temurin, Zulu, Corretto...).
actions/setup-node@v4	Instala Node.js y npm.
actions/cache@v4	Guarda en caché dependencias (Maven, npm...) para acelerar los builds.
actions/upload-artifact@v4	Guarda ficheros del runner (JARs, informes de tests) para descargar después.

Las Actions oficiales (actions/...) son de confianza. Las de terceros ejecutan código en tu runner con acceso a tus secrets — revísalas antes de usarlas.

9.8 — Secrets: almacenar credenciales de forma segura

- Nunca escribas contraseñas o tokens directamente en el YAML — el fichero es público.
- Settings → Secrets and variables → Actions → New repository secret
- Nombre en mayúsculas por convenio: DEPLOY_TOKEN, MAVEN_PASSWORD...
- GitHub cifra el valor — nadie puede leerlo, ni tú, una vez guardado.

The screenshot shows the GitHub interface for managing secrets and variables. The top navigation bar includes links for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings (which is highlighted). The left sidebar contains a list of settings categories: General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Rules, Actions, Webhooks, Copilot, Environments, Codespaces, Pages, Security and quality, and Advanced Security. The main content area is titled 'Actions secrets and variables' and contains two tabs: 'Secrets' (selected) and 'Variables'. Under the 'Secrets' tab, there are two sections: 'Environment secrets' and 'Repository secrets'. Both sections indicate that there are no secrets currently stored and provide a button to manage or create new secrets.

< > Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

This repository has no secrets.

New repository secret

9.8 — Secrets: cómo usarlos en el workflow

La sintaxis `${{ secrets.NOMBRE }}` sustituye el valor en tiempo de ejecución. En los logs aparece como `***` aunque alguien intente imprimirlo con `echo`.

```
- name: Publicar en el registro
  run: mvn deploy
  env:
    MAVEN_USERNAME: ${ secrets.MAVEN_USERNAME }
    MAVEN_PASSWORD: ${ secrets.MAVEN_PASSWORD }
```

Los secrets NO están disponibles en PRs de forks externos. Esto evita que alguien cree un fork, añada `echo ${ secrets.TOKEN }` al workflow y extraiga tus credenciales.

✓ Ideas clave

- CI — ejecutar comprobaciones automáticas en cada cambio para detectar errores antes de que lleguen a main.
- GitHub Actions — CI/CD integrado en GitHub. Se configura con ficheros YAML en `.github/workflows/`.
- Event → Workflow → Job → Steps — cada push activa el workflow, que ejecuta jobs con steps en orden.
- Si el workflow falla, el log indica exactamente qué test falló y con qué valores (expected: <5> but was: <-1>).
- Secrets — almacenan credenciales cifradas. Nunca escribas contraseñas directamente en el YAML.
- if: always() garantiza que un step se ejecute aunque los anteriores fallen — imprescindible para guardar informes de tests.